Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

PRÁCTICA 2 ALGORÍTMICA GRUPO A3 LIANA



UNIVERSIDAD
DE GRANADA

Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

Índice

1. **Explicación ejercicio 2.2 (Fuerza bruta)**
2. **Explicación ejercicio 2.2 (Divide y vencerás)**
3. **Discusión ejercicio 2.2**
4. **Explicación ejercicio 2.4 (Fuerza bruta)**
5. **Explicación ejercicio 2.4 (Divide y vencerás)**
6. **Discusión ejercicio 2.4**

1. Explicación ejercicio 2.2(Fuerza bruta)

El problema del máximo y segundo máximo consiste en encontrar los dos valores más grandes que están contenidos en una matriz, para el que en el pdf se nos ponen las siguientes restricciones:

- max1: el máximo valor de la matriz
- max2: el segundo máximo valor de la matriz
- max1 > max2

Para resolver el problema utilizando la fuerza bruta simplemente tenemos que hacer dos bucles anidados, uno para recorrer las filas y otro para las columnas. Dentro del bucle lo único que hay que hacer es un par de comprobaciones para en caso correcto sustituir los valores max1 y max2 por sus valores correspondientes.

Vamos a poner un ejemplo con un **caso de uso:**

Supongamos la siguiente matriz **M** de 2x3:
Fila 0: {3 8 5}
Fila 1: {10 2 7}

El algoritmo comienza inicializando **max1 en 3** y el **max2 en -1**.

Iteramos con **i = 0** (primera fila):

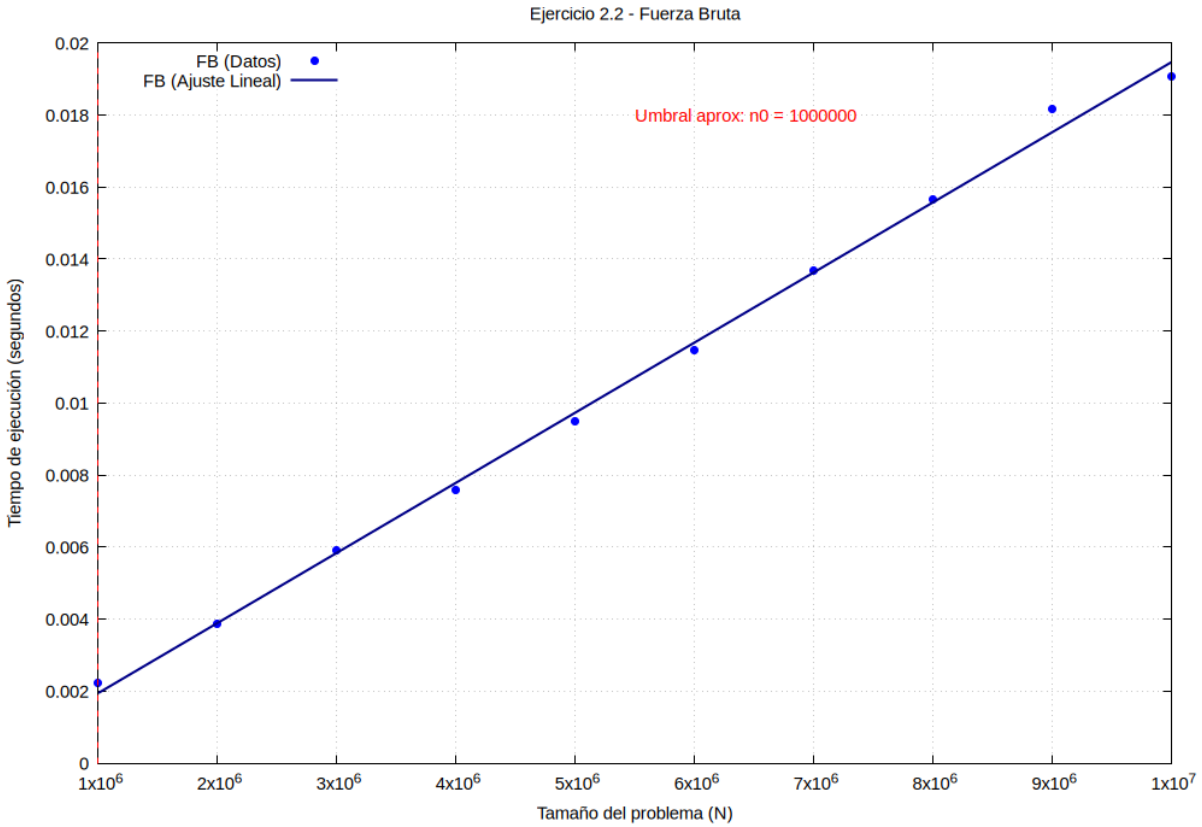
- **j = 0 (valor 3)**: Es el primer elemento, el máximo se mantiene en 3.
- **j = 1 (valor 8)**: Como 8 > 3, el 8 pasa a ser el máximo y el 3 baja a ser el segundo máximo.

- **j = 2 (valor 5)**: Como 5 es menor que 8 pero mayor que 3, el 5 pasa a ser el nuevo segundo máximo.

Iteramos con **i = 1** (segunda fila):

- **j = 0 (valor 10)**: Como 10 > 8, el 10 pasa a ser el máximo y el 8 (que era el máximo anterior) baja a ser el segundo máximo.
- **j = 1 (valor 2)**: Como 2 es menor que el segundo máximo (8), no ocurre nada.
- **j = 2 (valor 7)**: Como 7 es menor que el segundo máximo (8), no ocurre nada.

El resultado final sería un par con el **máximo 10** y el **segundo máximo 8**.



Eficiencia teórica: para saber el orden de eficiencia teórica que tiene nuestro programa nos vamos a fijar en el código teniendo en cuenta que **filas x columnas = N**.
Dentro de este tenemos una estructura de dos bloques anidados para las filas y columnas respectivamente, lo que nos da ya un órden de eficiencia de O(n). En el bucle tenemos un recorrido único, es decir, cada casilla de la matriz únicamente se comprueba una vez, por lo que no sumamos nada a la eficiencia. Por último, obviando por supuesto las operaciones básicas que hay dentro y fuera del bucle debido a que son de tiempo constante, se nos queda que el algoritmo utilizado tiene un orden de eficiencia de O(n).

Eficiencia empírica: se ve bastante claro en la gráfica de arriba que nuestro conjunto de valores de ejecución, el cual lo dejo abajo, prácticamente coincide con la gráfica teórica de órden de eficiencia de O(n). Aunque podemos apreciar una breve desviación con respecto al

órden teórico, este es despreciable pues depende de lo eficiente que sea la arquitectura en la que se ejecute el código.

Tamaño (N)	Tiempo (s)
1000000	**0,00224417**
2000000	**0,00388133**
3000000	**0,00592453**
4000000	**0,00759863**
5000000	**0,00947831**
6000000	**0,0114612**
7000000	**0,0136786**
8000000	**0,0156648**
9000000	**0,018148**
10000000	**0,0190695**

2. Explicación ejercicio 2.2(Divide y vencerás)

Ahora vemos este ejercicio desde un enfoque distinto utilizando recursividad para dividir el ejercicio en pequeñas partes usando la recursividad.

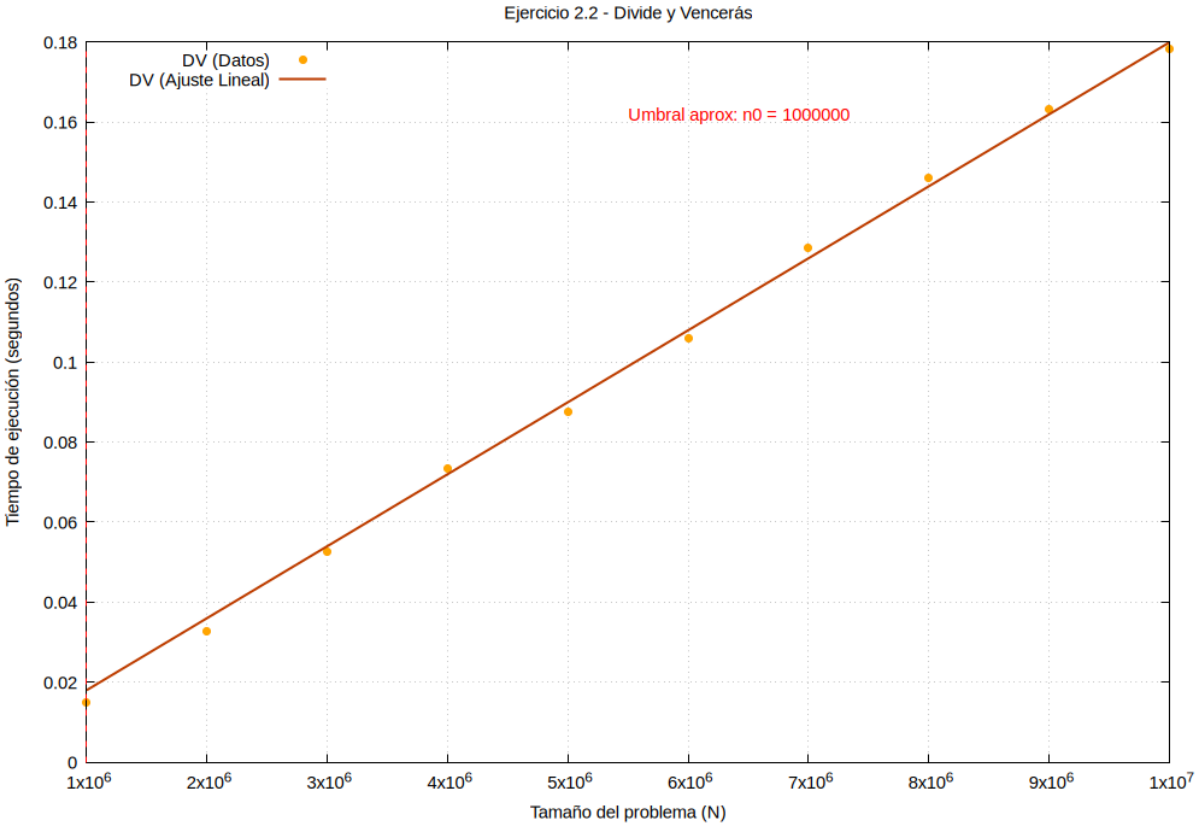
El **caso base** consiste en comprobar si **inicio_i == final_i** y **inicio_j == final_j** (una sola celda), y en caso de que esto ocurra se devuelve un par donde el primer elemento es el número de la celda y el segundo el valor mínimo posible.
La **división** se da cuando el caso base no es cierto, entonces dividimos la matriz en cuatro y hacemos recursividad con esas submatrices.
En la parte de **combinación**, ayudándonos de nuestra función auxiliar **calcularMaximo** comprobando y actualizando los valores de los máximos para las 4 submatrices comentadas anteriormente.

Vamos a poner un ejemplo con un **caso de uso:**

Supongamos una matriz M 2x2:
Fila 0: {10 5}
Fila 1: {3 8}

El algoritmo divide la matriz hasta que se llega a los casos base:
(0, 0): devuelve el par {10, INT_MIN}.
(1, 1): devuelve el par {8, INT_MIN} y la función **calcularMáximo** los compara dando como resultado **{10, 8}**.
(1, 0): devuelve el par {3, INT_MIN} y la función **calcular Máximo** no cambia nada quedándose el resultado **{10, 8}**.
(0, 1): devuelve {5, INT_MIN} y la función **calcular Máximo** tampoco cambia nada dejando el resultado **{10, 8}**.

Al final del algoritmo se nos queda que **max1 = 10** y **max2 = 8**.



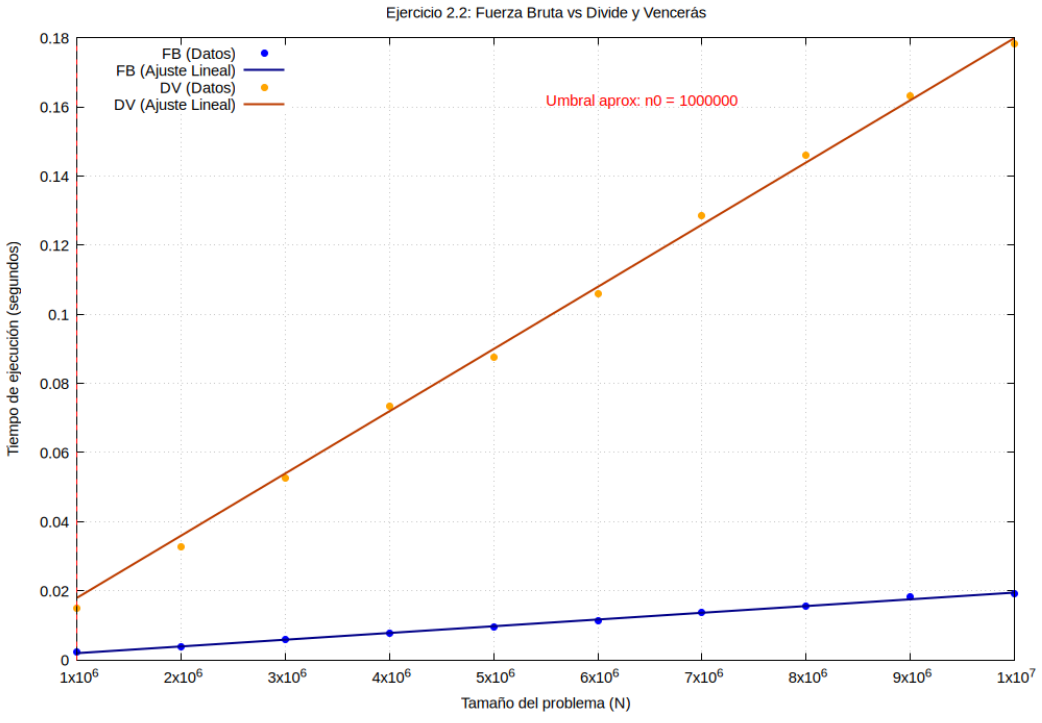
Eficiencia teórica: dando por hecho al igual que para fuerza bruta que **N = filas x columnas,** vamos a fijarnos en nuestro código para calcular el órden de eficiencia.
Al igual que pasaba en el de fuerza bruta, cada elemento de la matriz únicamente es visitado una vez, por lo que omitiendo el resto de operaciones elementales y fijándonos únicamente en los bucles del código podemos obtener que el algoritmo tiene un orden de eficiencia de **O(n)**.

Eficiencia empírica: al igual también que en el de fuerza bruta, como podemos observar en el ajuste gráfico que hemos hecho arriba, nuestro conjunto de datos es muy similar a la función que representa el orden de eficiencia teórico, por lo que viendo esto y fijándonos en que los datos crecen de una forma proporcional al tamaño de la entrada podemos deducir también que el orden de eficiencia empírico será de **O(n)**.

Tamaño (N)	Tiempo (s)
1000000	**0,0150484**
2000000	**0,0326737**
3000000	**0,0524529**
4000000	**0,0733518**
5000000	**0,0875599**
6000000	**0,106046**
7000000	**0,128537**
8000000	**0,146104**
9000000	**0,163148**
10000000	**0,178383**

3. Discusión ejercicio 2.2

Para comenzar la comparación vamos a ver primero las dos gráficas juntas para que todo sea más visual.



Como podemos observar, en este ejercicio el enfoque de fuerza bruta es mucho más eficiente que el de divide y vencerás aunque como hemos visto antes ambos tienen el mismo orden de eficiencia.
De primeras esto no aparenta tener mucho sentido pero tiene una explicación muy sencilla, y es que nosotros únicamente hemos tenido en cuenta la eficiencia temporal dejando de lado la espacial.
Cuando la matriz comienza a tener un número de elementos elevado, se hace una cantidad enorme de llamadas recursivas, lo cual también implica que en cada llamada haya que guardar el estado actual de la memoria en la pila. Al contrario, el de fuerza bruta lo único que hace es recorrer la matriz elemento por elemento y hacer una comparación cuyo tiempo de ejecución es despreciable.
Por tanto, realmente el de divide y vencerás en este ejercicio no tiene mucho sentido porque hace exactamente lo mismo que el de fuerza bruta pero teniendo que hacer mucho más trabajo por cada celda leída.

4. Explicación ejercicio 2.4 (Fuerza bruta)

El problema de las inversiones consiste en tener un vector y calcular el número de inversiones que contiene , para ello sabemos que una inversión cumple estos dos requisitos como nos dice el pdf:

- i<j
- v[i] > v[j]

Para resolver el problema mediante la técnica de fuerza bruta, hemos diseñado un algoritmo que explora de manera exhaustiva todas las soluciones.La lógica consistirá en evaluar todos los pares posibles de elementos dentro del vector para comprobar si cumplen la condición de inversión , por eso usamos dos bucles con dos iteradores el primero que es i recorre el vector y el segundo que es j recorre el resto del vector desde la siguiente posición.

Un ejemplo de **pseudocódigo** de esta solución sería este:

```
Función InversionesFB(v: vector de enteros)
  contador = 0
  tam = tamaño de v

  Para i desde 0 hasta tam - 1 hacer:
    Para j desde i + 1 hasta tam - 1 hacer:
      Si v[i] > v[j] entonces:
        contador <- contador + 1

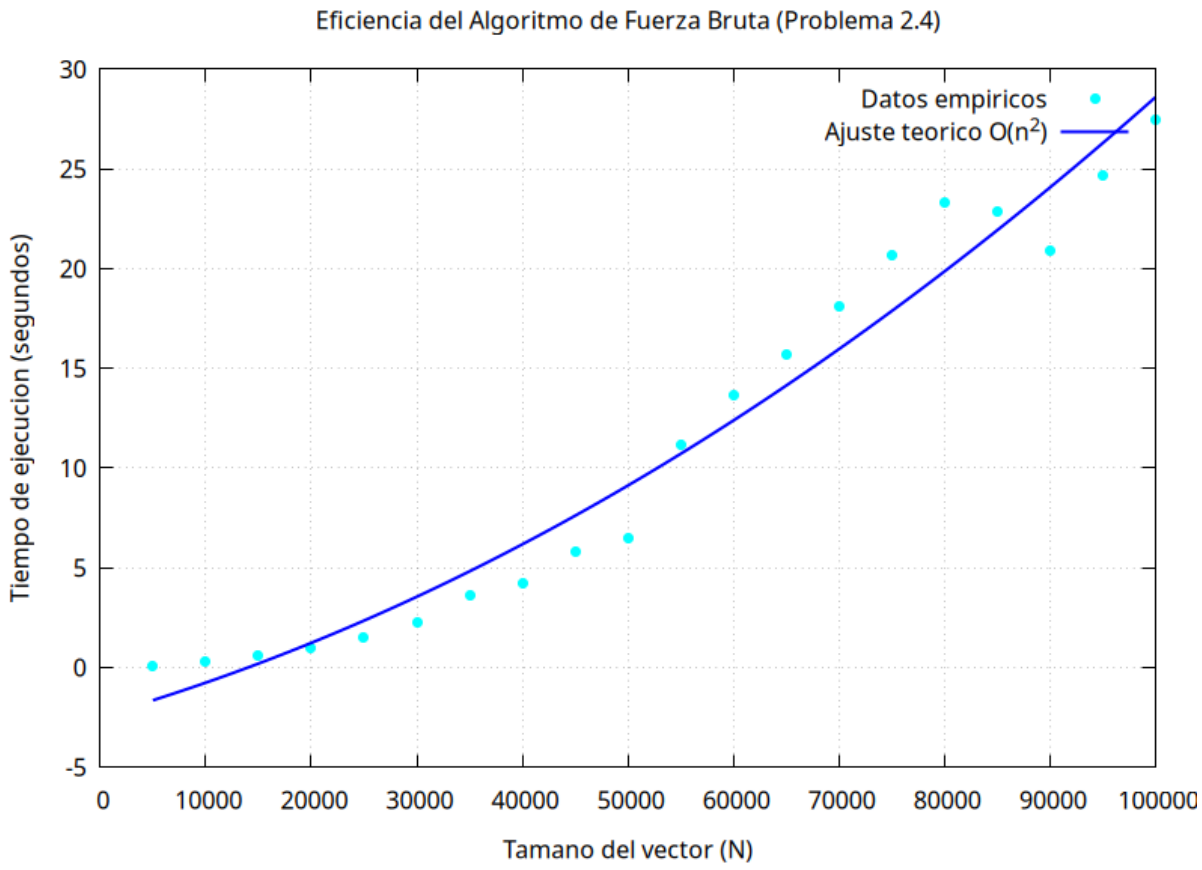
        Devolvemos el contador
Fin Función
```

Vamos a poner un ejemplo con un **caso de uso**:

Imagina que tenemos el vector **v1 = {3,1,4,2}**

iteramos con i = 0 y mientras j = 1 (3 > 1 , habría una inversión y aumenta el contador) , después j = 2 (3 < 4 , no pasaría nada) , j = 3 (3 > 2 , hay inversión , sumamos 1 al contador).
iteramos con i = 1 y mientras j = 2 (1 < 4 , no pasa nada) , j = 3 (1 < 2 , no hay inversión).
iteramos con i = 2 y mientras j = 3 (4 > 2 , hay inversión , sumamos una al contador).

El contador quedaría con un total de **3 inversiones**.



Eficiencia Teórica: Para saber cuánto va a tardar nuestro algoritmo, solo tenemos que mirar cómo están construidos los bucles en el código. Hemos utilizado dos bucles anidados(un for dentro de otro for).El primer bucle for pasa por cada número del vector. Por cada uno de esos números, el segundo bucle for revisa todos los que quedan a la derecha.
Si sumamos todas las comparaciones que hace, nos da una fórmula matemática de $2n(n-1)$ operaciones. Por tanto nos quedamos con la parte que más crece y decimos que el tiempo de ejecución aumenta de una forma cuadrática,ya que su eficiencia es **$O(n^2)$.** Además, como

no creamos ningún vector nuevo por dentro y solo usamos una variable para contar, casi que no gastamos memoria extra,así que su complejidad espacial es constante **O(1)**.

Eficiencia Empírica: La teoría que acabamos de explicar se comprueba perfectamente en la práctica si nos ponemos a mirar la gráfica de arriba. Los puntos celestes representan el tiempo real que ha tardado nuestro ordenador en resolver vectores cada vez más grandes.

Vemos que el tiempo no crece en una línea recta. Al principio (con 5.000 o 10.000 datos) tarda muy poco, pero según nos acercamos a los 100.000 elementos,el tiempo se dispara formando una curva pronunciada hacia arriba, rozando los 28 segundos.La línea azul que pasa por encima de los puntos es un ajuste matemático de una parábola. Queda por tanto demostrado que nuestro programa tiene una eficiencia cuadrática **O(n^2).**

Elementos	Tiempo de ejecución
10000	**0.285791**
15000	**0.572563**
20000	**1.01261**
25000	**1.47372**
30000	**2.25788**
35000	**3.63843**
40000	**4.25977**
45000	**5.82353**
50000	**6.52156**
55000	**11.1473**
60000	**13.68**
65000	**15.7101**
70000	**18.1078**
75000	**20.685**
80000	**23.3467**
85000	**22.8852**
90000	**20.9213**
95000	**24.646**
100000	**27.4372**

5. Explicación ejercicio 2.4 (Divide y Vencerás)

Para resolver este mismo problema de forma mucho más rápida,hemos utilizado la técnica de Divide y Vencerás.En vez de comparar todos los números con todos, nuestro algoritmo se basa en el famoso método de ordenación MergeSort.

La lógica consiste en dividir el vector por la mitad una y otra vez hasta que nos queden trozos de un solo elemento.Después empezamos a juntar esos trozos ordenándolos al mismo tiempo. La ventaja está en el momento de la mezcla, si al comparar la mitad izquierda con la mitad derecha vemos que un número de la derecha es menor que uno de la izquierda, pues tenemos una inversión. Y no solo eso: como la mitad izquierda ya está ordenada por dentro, ese número de la derecha será menor que *todos* los números que queden por revisar en la izquierda, lo que nos permite sumar muchas inversiones de golpe sin tener que mirarlas una a una.

Un ejemplo de pseudocódigo de esta solución sería este:

```
Función inversiones(v, inicio, fin)
  contador = 0
  Si inicio < fin entonces:
    medio = (inicio + fin) / 2
    contador += Inversiones(v, inicio, medio)      // Mitad izquierda //
    contador += Inversiones(v, medio + 1, fin)     // Mitad derecha //
    contador += ContarAlMezclar(v, inicio, medio, fin) // Mezclamos y contamos //
  Devolver contador
Fin Función
Función contInversiones(v, inicio, medio, fin)
  contador = 0
  Mientras queden elementos en ambas mitades:
    Si elemento_izq <= elemento_der:
      Lo guardamos en el vector auxiliar (no hay inversión)
    Si no entonces:
      Lo guardamos en el vector auxiliar
```

contador += (medio - indice_izq + 1)

Copiamos los elementos restantes y actualizamos el vector original

Devolver contador

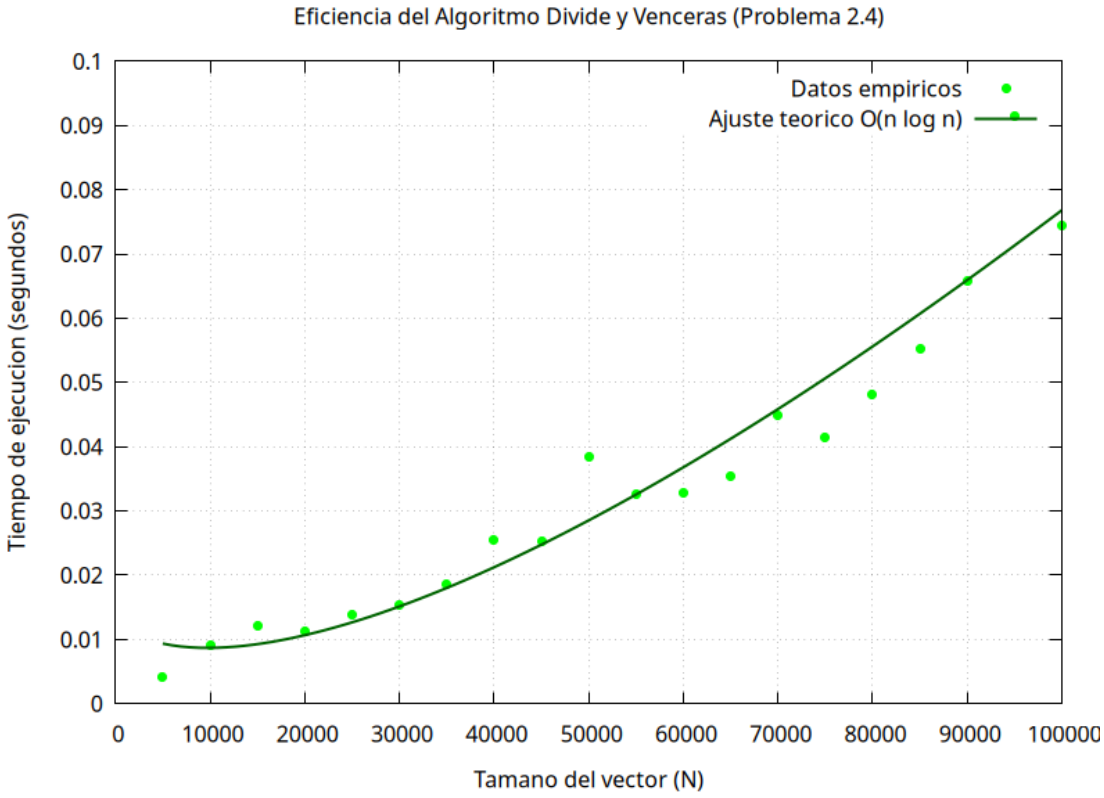
Fin Función

Vamos a poner un ejemplo con un **caso de uso**:

Imagina que tenemos otra vez el vector **v1 = {3,1,4,2}**

Dividimos por la mitad y se queda dos mitades {**3,1} y {4,2}** , comparamos el 3 y el 1 como el 1 es menor hay una inversión por tanto sumamos 1 y se ordena el vector **{1,3}** , hacemos lo mismo con la otra parte y se suma otro a la inversión y se queda **{ 2,4}** , ahora tenemos que mezclar al final **{1,3} ,**
{2 ,4} comparas el 1 y el 2 y no hay problemas , comparas el 3 y el 2 y hay una inversión y ya no hay más inversiones.

El contador final suma también **3 inversiones**, igual que en la fuerza bruta, pero haciendo menos pasos.



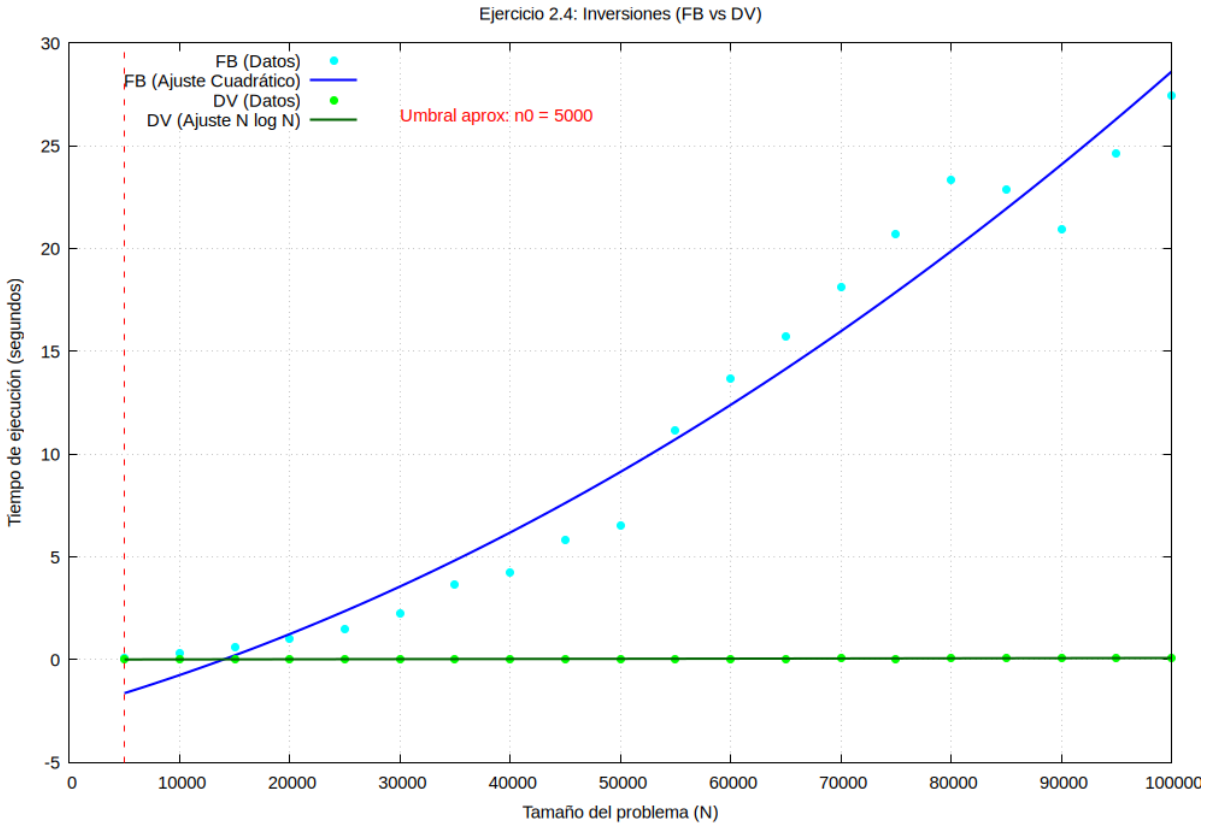
Eficiencia Teórica: Al analizar nuestro algoritmo, vemos que la parte de dividir el vector por la mitad varias veces sigue un patrón logarítmico. Por otro lado, vemos que la función que

mezcla y cuenta recorre todos los elementos de esas mitades en tiempo lineal. Al combinar ambos procesos, el tiempo de ejecución crece a un ritmo de O(nlogn). Sin embargo, a diferencia de la fuerza bruta, aquí sí necesitamos gastar memoria extra, ya que creamos un vector auxiliar para ir guardando los números ordenados mientras los vamos mezclando.Por tanto, su eficiencia linealmente: O(n).

Eficiencia Empírica:Si miramos la gráfica comparativa (la línea verde oscura) o los datos obtenidos, vemos que para ordenar 100.000 elementos, el algoritmo Divide y Vencerás ha tardado apenas 0.074 segundos. Si recordamos que la Fuerza Bruta tardaba casi 28 segundos para ese mismo tamaño, la diferencia es bastante notable.En la gráfica su comportamiento es O(nlogn).

Elementos	Tiempo de ejecución
10000	**0.009034**
15000	**0.012228**
20000	**0.011239**
25000	**0.013847**
30000	**0.015411**
35000	**0.018551**
40000	**0.025451**
45000	**0.025261**
50000	**0.038508**
55000	**0.032602**
60000	**0.032738**
65000	**0.035477**
70000	**0.044829**
75000	**0.04154**
80000	**0.048033**
85000	**0.05517**
90000	**0.065804**
95000	**0.091477**
100000	**0.074468**

6. Discusión del ejercicio 2.4



Tras haber analizado los dos algoritmos por separado. Si observamos la gráfica comparativa donde hemos juntado ambas ejecuciones bajo la misma escala de tiempo, el contraste es bastante grande.

Por un lado, los datos de la **Fuerza Bruta (puntos celestes)** se disparan rápidamente rozando los 28 segundos para unos 100.000 elementos, demostrando su ineficiencia cuadrática O(n2).Por el otro lado tenemos ,los datos del **Divide y Vencerás (puntos verdes)** ni siquiera se despegan del eje horizontal.Su crecimiento logarítmico O(nlogn) es muy rápido que su línea parece plana.

El umbral de rentabilidad es el tamaño exacto de datos a partir del cual un algoritmo más rápido en teoría empieza a ganarle en la práctica a uno que es más lento. En nuestra gráfica lo hemos señalado con una línea vertical roja en los 5.000 elementos. Si miramos los datos reales, para un vector de 5.000 números la Fuerza Bruta ha tardado unos 0.085 segundos,mientras que el Divide y Vencerás lo ha resuelto en solo unos 0.004 segundos.Esto

nos dice que a partir de este punto, existe una clara diferencia y mejora con el Divide y Vencerás.

Ventajas y desventajas de cada algoritmo: Como la conclusión final al problema,podemos destacar las siguientes ventajas y desventajas de usar un algoritmo u otro:

Fuerza Bruta:

- **Ventajas:** El código es muy intuitivo y sencillo de imaginar. Aunque su mayor punto a favor es que no gasta memoria extra; hace las comprobaciones sobre el vector original usando solo una variable para contar, por lo que su eficiencia espacial es constante **O(1)**.
- **Desventajas:** Su tiempo de ejecución es **O(n2)** lo hace muy poco eficiente para procesar cantidades medianas o grandes de datos.

Divide y Vencerás:

- **Ventajas:** Su grandísima ventaja es la eficiencia **(O(nlogn))**. Es un algoritmo que nos permite resolver problemas gigantescos en muy poco tiempo.
- **Desventajas:** Es bastante más complicado de imaginar. Aunque su peor punto es el gasto de memoria ya que para poder ordenar y contar necesitamos crear un vector auxiliar que va copiando los datos, lo que hace que su complejidad espacial suba a **O(n)**.